# CONSTITUTION

## INHERITED FROM Helix Constitution

This module is a submodule of a Helix-family project (e.g. HelixCode, HelixAgent, ATMOSphere) that includes the Helix Constitution submodule at the parent's `constitution/` path. All rules in `constitution/CLAUDE.md` and the `constitution/Constitution.md` it references (universal anti-bluff covenant §11.4, no-guessing mandate §11.4.6, credentials-handling mandate §11.4.10, host-session safety §12, data safety §9, mutation- paired gates §1.1) apply unconditionally to every change landed here. The module-specific rules below extend them — they never weaken any universal clause.

When this file disagrees with the constitution submodule, the constitution wins. Locate the constitution submodule from any arbitrary nested depth using its `find_constitution.sh` helper.

Canonical reference: https://github.com/HelixDevelopment/Helix-Constitution

---

# HelixCode Constitution

> **Inherits from:** `HelixConstitution/Constitution.md` — all universal clauses in that document apply unconditionally. Rules below extend or tighten them; they MUST NOT weaken any inherited clause.

## HelixCode Project Constitution

**Version**: 1.0.0 **Effective Date**: 2026-04-30 **Scope**: This Constitution applies to HelixCode and ALL its submodules **Authority**: Cascaded from HelixAgent root governance with HelixCode-specific addenda

---

# Preamble

HelixCode is an enterprise-grade distributed AI development platform. This Constitution establishes the non-negotiable rules that govern all development, testing, deployment, and maintenance activities within the project. Every contributor, agent, and automated process MUST adhere to these rules. No exceptions.

---

# CONST-001: No CI/CD Pipelines (Permanent)

No `.github/workflows/`, `.gitlab-ci.yml`, `Jenkinsfile`, `.travis.yml`, `.circleci/`, or any automated pipeline. No Git hooks. All builds and tests run manually or via Makefile/script targets.

**Rationale**: Manual execution ensures human oversight and prevents automated propagation of bluffs.

---

# CONST-002: No Mocks in Production (Permanent)

## CONST-002a: Production Code

Mocks, stubs, fakes, placeholder classes, TODO implementations are STRICTLY FORBIDDEN in production code. All production code is fully functional with real integrations.

## CONST-002b: Test Code

Mocks/stubs/fakes MAY be used ONLY in unit tests (files ending `_test.go` run under `go test -short`).

**Rationale**: Production bluffs have repeatedly been discovered where features appeared implemented but were non-functional.

---

## CONST-003: No HTTPS for Git (Permanent)

SSH URLs only (`git@github.com:…`, `git@gitlab.com:…`, etc.) for clones, fetches, pushes, and submodule updates. SSH keys are configured on every service.

## <<<<<<< HEAD

## CONST-004: No Manual Container Commands (Permanent)

Container orchestration is owned by the project's binary/orchestrator (e.g., `make build` → `./bin/<app>`). Direct `docker`/`podman start|stop|rm` and `docker-compose up|down` are prohibited as workflows.

---

## CONST-005: 100% Real Data for Non-Unit Tests

Beyond unit tests, all components MUST use actual API calls, real databases, live services. No simulated success. Fallback chains tested with actual failures.

**Verification**: Every integration/E2E test MUST connect to real services or skip (not fail) if unavailable.

---

## CONST-006: Challenge Coverage (Permanent)

Every component MUST have Challenge scripts (`./challenges/scripts/`) validating real-life use cases. No false success — validate actual behavior, not return codes.

---

## CONST-007: Health & Observability

Every service MUST expose health endpoints. Circuit breakers for all external dependencies. Prometheus / OpenTelemetry integration where applicable.

---

## CONST-008: Documentation & Quality

Update `CLAUDE.md`, `AGENTS.md`, and relevant docs alongside code changes. Pass language-appropriate format/lint/security gates. Conventional Commits: `<type>(<scope>): <description>`.

---

## CONST-009: Validation Before Release

Pass the project's full validation suite (`make ci-validate-all`-equivalent) plus all challenges (`./challenges/scripts/run_all_challenges.sh`).

---

## CONST-010: Comprehensive Verification

Every fix MUST be verified from all angles: runtime testing (actual HTTP requests / real CLI invocations), compile verification, code structure checks, dependency existence checks, backward compatibility, and no false positives. Grep-only validation is NEVER sufficient.

---

## CONST-011: Resource Limits for Tests & Challenges

ALL test and challenge execution MUST be strictly limited to 30-40% of host system resources. Use `GOMAXPROCS=2`, `nice -n 19`, `ionice -c 3`, `-p 1` for `go test`. Container limits required.

---

## CONST-012: Bugfix Documentation

All bug fixes MUST be documented in `docs/issues/fixed/BUGFIXES.md` with root cause analysis, affected files, fix description, and a link to the verification test/challenge.

---

## CONST-013: Real Infrastructure for All Non-Unit Tests

Mocks/fakes/stubs/placeholders MAY be used ONLY in unit tests. ALL other test types — integration, E2E, functional, security, stress, chaos, challenge, benchmark, runtime verification — MUST execute against REAL running systems with REAL containers, REAL databases, REAL services, and REAL HTTP calls.

---

## CONST-014: Reproduction-Before-Fix (Mandatory)

Every reported error, defect, or unexpected behavior MUST be reproduced by a Challenge script BEFORE any fix is attempted. Sequence:

1. Write the Challenge first
2. Run it; confirm fail (it reproduces the bug)
3. Then write the fix
4. Re-run; confirm pass
5. Commit Challenge + fix together

The Challenge becomes the regression guard for that bug forever.

---

## CONST-015: Concurrent-Safe Containers

Any struct field that is a mutable collection (map, slice) accessed concurrently MUST use thread-safe primitives. Bare `sync.Mutex + map/slice` combinations are prohibited for new code.

---

## CONST-016: Definition of Done (Universal)

A change is NOT done because code compiles and tests pass. "Done" requires pasted terminal output from a real run.

- **No self-certification**: Words like *verified, tested, working, complete, fixed, passing* are forbidden in commits/PRs/replies unless accompanied by pasted output from a command that ran in that session.

- **Demo before code**: Every task begins by writing the runnable acceptance demo
- **Real system, every time**: Demos run against real artifacts
- **Skips are loud**: `t.Skip` without a trailing `SKIP-OK: #<ticket>` comment breaks validation

---

# CONST-035 — Anti-Bluff Tests & Challenges (User-Mandate Forensic Anchor)

### §11.9 User-Mandate Forensic Anchor (2026-04-29)

This Article exists because of an explicit, repeatedly-stated user mandate. The verbatim text:

> "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

This anchor is the primary authority for the entire Article. The operative rule is:

**The bar for shipping is not "tests pass" but "users can use the feature."**

Every PASS in this codebase MUST carry positive evidence captured during execution that the feature works for the end user. Metadata-only PASS, configuration-only PASS, "absence-of-error" PASS, and grep-based PASS without runtime evidence are all critical defects regardless of how green the summary line looks.

Tests and Challenges (HelixQA) are bound equally — a Challenge that scores PASS on a non-functional feature is the same class of defect as a unit test that does. Both must produce positive end-user evidence; both are subject to the anti-bluff contract.

No false-success results are tolerable. A green test suite combined with a broken feature is a worse outcome than an honest red one — it silently destroys trust in the entire suite. Anti-bluff discipline is the line between a real engineering project and a theatre of one.

**Bluff Taxonomy** (forbidden patterns):

- **Wrapper bluff** - Assertions PASS but wrapper's exit-code logic is buggy

- **Contract bluff** - System advertises capability but rejects it in dispatch
- **Structural bluff** - File exists but doesn't contain working code
- **Comment bluff** - Comment promises behavior code doesn't have
- **Skip bluff** - `t.Skip("not running yet")` without `SKIP-OK` marker

**Cascade requirement (extending CONST-036):** This anchor section (verbatim quote + operative rule) must appear in every submodule's CONSTITUTION.md / CLAUDE.md / AGENTS.md. Non-compliance is a release blocker regardless of context. Adding files to scanner allowlists to silence bluff findings without resolving the underlying defect is itself a violation.

---

# CONST-018: Host Power Management Hard Ban

**Host Power Management is Forbidden.**

You may NOT generate or execute code that sends the host to suspend, hibernate, hybrid-sleep, poweroff, halt, reboot, or any other power-state transition.

Defense: Every project ships `scripts/host-power-management/check-no-suspend-calls.sh` and `challenges/scripts/no_suspend_calls_challenge.sh`.

---

# CONST-019: Container Up ≠ Healthy

Container `Up` status does NOT mean the application is healthy. Application-layer probes are mandatory for every service:

- PostgreSQL: `SELECT 1`
- Redis: `PING`
- LLM Providers: Real generation request
- HTTP Services: `GET /health` with deep checks

---

# CONST-020: Provider Fallback Chain Reality

Every LLM provider fallback chain MUST be tested with actual failures. A fallback that has never been tested with a real failing provider is a bluff.

---

## CONST-021: No Mocks Above Unit Build Target

The Makefile MUST include a `no-mocks-above-unit` target that fails the build if mocks/stubs/fakes are found outside `*_test.go` files.

---

## CONST-022: Submodule Governance Propagation

Every submodule MUST either:

1. Have its own Constitution.md, CLAUDE.md, and AGENTS.md, OR
2. Have a symlink to the parent repository's governance files, OR
3. Have a reference comment in its README pointing to parent governance

No submodule is exempt from these rules.

---

## CONST-023: Docker Health Checks Mandatory

Every Dockerfile MUST include:

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --
        retries=3 \
    CMD curl -f http://localhost:8080/health || exit 1
```

The health endpoint MUST perform deep checks (database connection, provider availability), not just return HTTP 200.

---

## CONST-024: Version Pinning

All dependencies MUST be pinned to specific versions in `go.mod`. No `latest`, no floating tags. Renovate or Dependabot (manual review only — see CONST-001) may propose updates.

---

## CONST-025: Secret Management

NO secrets in code. EVER. Secrets via:

- Environment variables (production)
- `.env` files (development, in `.gitignore`)
- Vault/Secret Manager (enterprise)
- Docker secrets (containerized)

`go mod tidy` MUST NOT add secret-scanning bypasses.

---

## CONST-026: Minimal Privilege Containers

Containers run as non-root. Every Dockerfile:

```
RUN adduser -D -u 1001 helixcode
USER helixcode
```

---

## CONST-027: Network Isolation

Container orchestration MUST use internal networks. Services communicate via named hosts, not exposed ports where possible.

---

## CONST-028: Backup Before Destructive Operations

Every file editing tool MUST create backups before modification. The backup MUST be restorable.

---

## CONST-029: Input Validation at All Boundaries

Every public function MUST validate inputs. No trust of caller-provided data. SQL injection, path traversal, command injection MUST be impossible by design.

---

## CONST-030: Graceful Degradation

When external services are unavailable, the system MUST degrade gracefully:

- Return partial results where possible
- Queue operations for retry
- Inform user of degraded state
- NEVER crash or hang indefinitely

---

## CONST-031: Audit Trail

Every significant operation MUST be logged with:

- Timestamp
- User identity
- Operation type
- Success/failure status
- Resource affected

Log retention: 90 days minimum.

---

## CONST-032: Emergency Stop

Every long-running or distributed operation MUST support cancellation via `context.Context`. Users MUST be able to interrupt any operation.

---

## CONST-033: Data Integrity

Database writes MUST be transactional. Partial writes MUST be rolled back. Consistency checks MUST run periodically.

---

## CONST-034: API Stability

Public APIs maintain backward compatibility within major versions. Deprecation requires:

- 6-month notice
- Migration guide
- Compatibility shim

---

# CONST-035: End-User Usability Mandate (2026-04-29 Strengthening)

A test or Challenge that PASSES is a CLAIM that the tested behavior **works for the end user of the product**.

The HelixAgent project has repeatedly hit the failure mode where every test ran green AND every Challenge reported PASS, yet most product features did not actually work. This MUST NOT recur in HelixCode.

Every PASS result MUST guarantee: a. **Quality** - correct behavior under real inputs, edge cases, concurrency b. **Completion** - wired end-to-end with no stub/placeholder gaps c. **Full usability** - a user following documented request shapes SUCCEEDS

A passing test that doesn't certify all three is a **bluff** and MUST be tightened.

**Bluff taxonomy** (each pattern observed and now forbidden):

- **Wrapper bluff** - assertions PASS but wrapper's exit-code logic is buggy
- **Contract bluff** - system advertises capability but rejects it in dispatch
- **Structural bluff** - `check_file_exists` passes but doesn't run the test
- **Comment bluff** - comment promises behavior code doesn't actually have
- **Skip bluff** - `t.Skip("not running yet")` without `SKIP-OK: #<ticket>` marker

**Full background**: `docs/HOST_POWER_MANAGEMENT.md` and this Constitution (CONST-035).

---

# CONST-036: Propagation to Submodules

This Constitution, along with CLAUDE.md and AGENTS.md, MUST be propagated to ALL submodules. Each submodule's governance MUST reference this parent Constitution. Changes to this Constitution MUST trigger review of all submodule governance files.

---

# CONST-037: LLMsVerifier Single Source of Truth Mandate

**Rule**: LLMsVerifier SHALL BE the sole authoritative source for:

1. All model metadata (names, IDs, context windows, capabilities)
2. All provider metadata (endpoints, auth types, supported models)
3. All verification status (verified, partial, failed, pending)
4. All scoring data (overall scores, capability scores, tier rankings)
5. All rate-limit and cooldown state

**Prohibition**: NO hardcoded model lists, NO hardcoded provider lists, NO simulated model discovery. Any code path that presents a model or provider listing to a user MUST fetch that listing from the LLMsVerifier subsystem or its cached replica.

**Anti-Bluff Verification**:

- The challenge script `challenges/scripts/verifier_hardcode_check.sh` MUST scan all Go source files for hardcoded model arrays.
- Any `[]string{"gpt-4", "claude-3"}` or equivalent literal in production code is a constitutional violation.
- The only permitted hardcoded data is the LLMsVerifier service endpoint URL and the list of verification test types.

**Enforcement**: `make test-complete` MUST include a test that asserts `ModelManager.GetAvailableModels()` returns at least as many models as the verifier's database contains for configured providers. A test that passes while the CLI shows a hardcoded list is a TEST BLUFF and violates CONST-035.

---

# CONST-038: Model Provider Anti-Bluff Guarantee

**Rule**: Every model displayed to an end user MUST have been verified by LLMsVerifier within the last `verification_timeout` period (default: 24h). Models older than this MUST display a "stale" indicator and be deprioritized.

**Prohibition Against Test Bluffing**:

- A unit test that mocks the verifier client and asserts `GetAvailableModels()` returns 3 models DOES NOT satisfy this rule.

- An integration test that starts the verifier server, performs real provider discovery, and confirms the model count matches the actual provider API response DOES satisfy this rule.
- The Makefile target `make test-verifier-integration` MUST exist and MUST run without mocks.

**The "Tests Pass But Features Don't Work" Guarantee**:

```
NO TEST MAY PASS UNLESS THE FEATURE IT TESTS IS DEMONSTRABLY
USABLE
BY AN END USER IN THE SAME BUILD.
```

- If `TestModelList` passes but `helixcode --list-models` shows hard-coded data, the test is a BLUFF.
- If `TestProviderHealth` passes but the health endpoint returns `200 OK` for a provider that is actually down, the test is a BLUFF.
- If `TestLLMGeneration` passes but `--prompt "hello"` returns a simulated string, the test is a BLUFF.
- Bluff tests MUST be rewritten or deleted. There is no "grandfather" exception.

**Evidence Standard**: Every test that claims to verify model/provider functionality MUST:

1. Call a real API endpoint or a real verifier database
2. Assert on response content that could only come from that real source
3. Include a test that runs the CLI binary with `--list-models` and checks output against verifier data

---

# CONST-039: Real-Time Model Status Accuracy

**Rule**: Model status (available, rate-limited, cooldown, offline, deprecated) displayed to users MUST reflect the actual state as known by LLMsVerifier within `max_staleness` seconds (default: 60s).

**Polling vs. Push**:

- If WebSocket/SSE push is unavailable, the system MUST poll LLMsVerifier at most every `status_poll_interval` (default: 30s).
- The TUI MUST display a "last updated" timestamp with every model listing.
- Models in "cooldown" or "rate-limited" state MUST show the estimated recovery time if known.

**Accuracy Verification**:

- Challenge script `challenges/scripts/model_status_accuracy_challenge.sh` MUST:
  1. Artificially rate-limit a provider by exhausting its quota
  2. Wait for the status to propagate to the verifier
  3. Check that `helixcode --list-models` shows the rate-limited status within 60s
  4. Check that `SelectOptimalModel()` no longer selects the rate-limited model

**Prohibition**: Status indicators that are "always green" or that lag >60s behind reality violate this rule.

---

# CONST-040: All Providers and Models Integration Mandate

**Rule**: HelixCode MUST integrate with ALL providers and models that LLMsVerifier supports, subject only to:

1. The provider being explicitly disabled in configuration (`enabled: false`)
2. The API key being absent and the provider requiring one
3. The provider being marked `deprecated` in the verifier database

**Minimum Provider Set** (SHALL NOT be reduced without constitutional amendment): | Provider | Auth Type | Required Env Var | |----------|-----------|-----------------| | OpenAI | API Key | `OPENAI_API_KEY` | | Anthropic | API Key / OAuth | `ANTHROPIC_API_KEY` | | Gemini | API Key | `GEMINI_API_KEY` | | DeepSeek | API Key | `DEEPSEEK_API_KEY` | | Groq | API Key | `GROQ_API_KEY` | | Mistral | API Key | `MISTRAL_API_KEY` | | xAI | API Key | `XAI_API_KEY` | | OpenRouter | API Key | `OPENROUTER_API_KEY` | | Ollama | Local | None (auto-detect) | | Llama.cpp | Local | None (auto-detect) |

**Integration Requirement**: For every provider in the minimum set:

- There MUST be a provider adapter file in `internal/llm/` or `internal/verifier/adapters/`
- There MUST be a `*_test.go` file with real API tests (skipped only if `HELIX_SKIP_LIVE_PROVIDER_TESTS` is set)
- There MUST be a challenge script in `challenges/scripts/`
- The model listing MUST include models from this provider when the provider is enabled

---

# CONST-041: MCP / LSP / ACP / Embedding / RAG / Skills / Plugins Integration Mandate

**Rule**: LLMsVerifier integration SHALL extend beyond basic model listing to cover ALL capability dimensions:

1. **MCP (Model Context Protocol)**: The verifier MUST report which models support MCP tool calling. HelixCode's MCP subsystem MUST consult verifier capability flags before selecting a model for tool-use tasks.

2. **LSP (Language Server Protocol)**: The verifier MUST report code-analysis capabilities. Models without `code_analysis` capability MUST NOT be selected for refactoring or debugging tasks.

3. **ACP (Agent Capability Protocol)**: The verifier MUST report multi-agent coordination support. Models with `supports_parallel_tool_use` MUST be preferred for ACP workflows.

4. **Embedding**: The verifier MUST report `supports_embeddings` for each model. The `CogneeConfig` embedding model selection MUST be verifier-aware.

5. **RAG (Retrieval-Augmented Generation)**: The verifier MUST report context-window sizes. RAG chunking strategies MUST adapt to the selected model's `context_window_tokens` as reported by the verifier.

6. **Skills / Plugins**: The verifier MUST track plugin compatibility. Models flagged `plugin_compatible` MUST be used when skill/plugin execution is required.

**Capability Checklist** (MUST be verified by challenge):

- [ ] MCP tool calling verified for at least 3 providers
- [ ] LSP code-analysis verified for at least 3 providers
- [ ] ACP parallel tool use verified for at least 2 providers
- [ ] Embedding generation verified for at least 2 providers
- [ ] RAG context-window adaptation verified
- [ ] Skills/plugin execution verified for at least 2 providers

**Prohibition**: Capability flags MUST NOT be hardcoded. The `Provider.GetCapabilities()` method MUST return data sourced from the verifier's `VerificationResult` fields.

---

# Article XII — Repository Safety

## §12.1 (CONST-042) — No-Secret-Leak

No API key, token, password, certificate, or other credential may be committed to any repository owned by HelixDevelopment or vasic-digital, transitively or otherwise. All secrets live in `.env` files (mode 0600) listed in `.gitignore`. Any leak — to git, logs, build artefacts, screenshots, or external services — is a release blocker until rotated and post-mortemed.

**Operational requirements:**

- Every repo must have `.env`, `.env.local`, `.env.*` (with `!.env.example` exception), `*.pem`, `*.key`, `*.crt`, `id_rsa*` in `.gitignore`.
- `scripts/scan-secrets.sh` (or equivalent) must run before every push; failing it blocks the push.
- API keys for development are sourced from the canonical `../helix_agent/.env` (mode 0600, never under git) and copied — never symlinked, never committed — into per-repo `.env` files.

**Cascade requirement:** This article must appear verbatim in every owned-by-us repository's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Owned-by-us repos are listed in `scripts/owned-repos.txt` (or, until that file exists, the meta-repo `propagate-governance.sh` script's submodule walk excluding third-party trees).

## §12.2 (CONST-043) — No-Force-Push

No force push, force-with-lease push, history rewrite, branch deletion of `main`/`master`, or upstream-overwriting operation may be performed without explicit, in-conversation user approval given for that specific operation. Authorization for one push does not extend to subsequent pushes. Bypassing hooks (`--no-verify`), signature verification (`--no-gpg-sign`), or protected-branch rules also requires explicit approval. This applies to every repository in the HelixDevelopment / vasic-digital stack.

**Operational requirements:**

- Local pre-push hook at `scripts/git-hooks/pre-push` (installed by `scripts/install-git-hooks.sh`) must reject `--force` / `--force-with-lease` unless `HELIX_FORCE_PUSH_APPROVED=1` is set.

- The hook is a courtesy gate; this constitutional clause is the actual contract.
- Regular non-force pushes of new commits to existing branches on already-configured remotes are PERMITTED without per-push approval, scoped to a programme/conversation in which the user has authorised the cadence.

# Cascade requirement: Same as §12.1 — verbatim, every owned-by-us repo's three governance files.

## CONST-035 — Anti-Bluff Tests & Challenges (mandatory; inherits from root)

Tests and Challenges in this submodule MUST verify the product, not the LLM's mental model of the product. A test that passes when the feature is broken is worse than a missing test — it gives false confidence and lets defects ship to users. Functional probes at the protocol layer are mandatory:

- TCP-open is the FLOOR, not the ceiling. Postgres → execute `SELECT 1`. Redis → `PING` returns `PONG`. ChromaDB → `GET /api/v1/heartbeat` returns 200. MCP server → TCP connect + valid JSON-RPC handshake. HTTP gateway → real request, real response, non-empty body.
- Container `Up` is NOT application healthy. A `docker/podman ps Up` status only means PID 1 is running; the application may be crash-looping internally.
- No mocks/fakes outside unit tests (already CONST-030; CONST-035 raises the cost of a mock-driven false pass to the same severity as a regression).
- Re-verify after every change. Don't assume a previously-passing test still verifies the same scope after a refactor.
- Verification of CONST-035 itself: deliberately break the feature (e.g. `kill <service>`, swap a password). The test MUST fail. If it still passes, the test is non-conformant and MUST be tightened.

# CONST-033 clarification — distinguishing host events from sluggishness

Heavy container builds (BuildKit pulling many GB of layers, parallel podman/docker compose-up across many services) can make the host **appear** unresponsive — high load average, slow SSH, watchers timing out. **This is NOT a CONST-033 violation.** Suspend / hibernate / logout are categorically different events. Distinguish via:

- `uptime` — recent boot? if so, the host actually rebooted.
- `loginctl list-sessions` — session(s) still active? if yes, no logout.
- `journalctl ... | grep -i 'will suspend\|hibernate'` — zero broadcasts since the CONST-033 fix means no suspend ever happened.
- `dmesg | grep -i 'killed process\|out of memory'` — OOM kills are also NOT host-power events; they're memory-pressure-induced and require their own separate fix (lower per-container memory limits, reduce parallelism).

A sluggish host under build pressure recovers when the build finishes; a suspended host requires explicit unsuspend (and CONST-033 should make that impossible by hardening `IdleAction=ignore` + `HandleSuspendKey=ignore` + masked `sleep.target`, `suspend.target`, `hibernate.target`, `hybrid-sleep.target`).

If you observe what looks like a suspend during heavy builds, the correct first action is **not** "edit CONST-033" but `bash challenges/scripts/host_no_auto_suspend_challenge.sh` to confirm the hardening is intact. If hardening is intact AND no suspend broadcast appears in journal, the perceived event was build-pressure sluggishness, not a power transition.

---

# Article XI §11.9 — Anti-Bluff Forensic Anchor (cascaded from parent CONSTITUTION.md)

> Verbatim user mandate (2026-04-29, reasserted multiple times across 2026-05): "*We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!*"

Operative rule: **The bar for shipping is not "tests pass" but "users can use the feature."** Every PASS in this codebase MUST carry positive runtime evidence captured during execution. Metadata-only / configuration-only / absence-of-error / grep-based PASS without runtime evidence are critical defects regardless of how green the summary line looks. No false-success results are tolerable.

This anchor MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md alongside CONST-047 — see the parent repository's `CONSTITUTION.md` for the full text.

---

# CONST-048: Full-Automation-Coverage Mandate (cascaded from constitution submodule §11.4.25)

> Verbatim user mandate (2026-05-15): "*Make sure that every feature, every functionality, every flow, every use case, every edge case, every service or application, on every platform we support is covered with full automation tests which will confirm anti-bluff policy and provide the proof of fully working capabilities, working implementation as expected, no issues, no bugs, fully documented, tests covered! Nothing less than this does not give us a chance to deliver stable product! This is mandatory constraint which MUST BE respected without ignoring, skipping, slacking or forgetting it!*"

No feature / functionality / flow / use case / edge case / service / application on any supported platform of HelixCode may be considered deliverable until covered by automation tests proving six invariants: (1) anti-bluff posture (CONST-035) with captured runtime evidence; (2) proof of working capability end-to-end on target topology (no mocks beyond unit tests — see CONST-050); (3) implementation matches documented promise; (4) no open issues/bugs surfaced — cross-checked against §11.4.15 / §11.4.16 trackers; (5) full documentation in sync per §11.4.12; (6) four-layer test floor per §1 (pre-build + post-build + runtime + paired mutation).

Consuming projects MUST publish a coverage ledger (feature × platform × invariant-1..6 × status) regenerated as part of the release-gate sweep. Gaps tracked per §11.4.15 (`UNCONFIRMED:` / `PENDING_FORENSICS:` / `OPERATOR-BLOCKED:` with §11.4.21 audit) — rows that quietly omit a platform are CONST-048 violations.

**Cascade requirement:** This anchor (verbatim or by `CONST-048` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a

§11.4 PASS-bluff at the release-gate layer. No escape hatch. See constitution submodule `Constitution.md` §11.4.25 for the full mandate.

# CONST-049: Constitution-Submodule Update Workflow Mandate (cascaded from constitution submodule §11.4.26)

> Verbatim user mandate (2026-05-15): "*Every time we add something into our root (constitution Submodule) Constitution, CLAUDE.MD and AGENTS.MD we MUST FIRST fetch and pull all new changes / work from constitution Submodule first! All changes we apply MUST BE commited and pushed to all constitution Submodule upstreams! In case of conflict, IT MUST BE carefully resolved! Nothing can be broken, made faulty, corrupted or unusable! After merging full validation and verification MUST BE done!*"

Before ANY modification to `constitution/Constitution.md`, `constitution/CLAUDE.md`, or `constitution/AGENTS.md`, the agent or operator MUST execute the following 7-step pipeline in order:

1. **Fetch + pull first** inside the constitution submodule worktree — every configured remote fetched, then `git pull --ff-only` (or `--rebase` if non-FF; NEVER `--strategy=ours` / `--allow-unrelated-histories` without explicit authorization).
2. **Apply the change** with §11.4.17 classification + verbatim mandate quote.
3. **Validate before commit** — `meta_test_inheritance.sh` (or equivalent), no merge-conflict markers, cross-file consistency.
4. **Commit + push to ALL upstreams** — governance files only (NEVER `git add -A`); push to every configured remote. One-upstream commit = CONST-049 violation (also CONST-038/§6.W and §2.1).
5. **Conflict resolution** preserving union of governance content. Force-push to bypass conflicts is FORBIDDEN (CONST-043 / §9.2).
6. **Post-merge validation** — `git submodule update --remote --init` + re-run cascade verifier (CONST-047) confirming the new clause reaches every owned submodule.
7. **Bump consuming project pointer** — `.gitmodules`-tracked submodule pointer advanced to the new constitution HEAD in the SAME commit as cascade work.

**Cascade requirement:** This anchor (verbatim or by `CONST-049` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a

force-push without CONST-043 / §9.2 authorization. No escape hatch. See constitution submodule `Constitution.md` §11.4.26 for the full mandate.

# CONST-050: No-Fakes-Beyond-Unit-Tests + 100%-Test-Type-Coverage Mandate (cascaded from constitution submodule §11.4.27)

> Verbatim user mandate (2026-05-15): "*Mocks, stubs, placeholders, TODOs or FIXMEs are allowed to exist ONLY in Unit tests! All other test types MUST interract with real fully implemented System! No fakes, empty implementations or bluffing is allowed of any kind! All codebase of the project MUST BE 100% covered with every supported test type: unit tests, integration tests, e2e tests, full automation tests, security tests, ddos tests, scaling tests, chaos tests, stress tests, performance tests, benchmarking tests, ui tests, ux tests, Challenges (fully incorporating our Challenges Submodule — https://github.com/vasic-digital/Challenges). EVERYTHING MUST BE tested using HelixQA (fully incorporating HelixQA Submodule — https://github.com/HelixDevelopment/HelixQA). HelixQA MUST BE used with all possible written tests suites (test banks) for every applications, service, platform, etc and execution of the full HelixQA QA autonomous sessions! All required dependency Submodules MUST BE added into the project as well (fully recursive!!!).*"

Two cooperating invariants:

**(A) No-fakes-beyond-unit-tests.** Mocks, stubs, fakes, placeholders, `TODO`, `FIXME`, "for now", "in production this would", or empty-implementation patterns are PERMITTED only in unit-test sources (`*_test.go` files invoked without the integration build tag; `HelixCode/tests/unit/`; etc.). Every other test type — integration, E2E, full automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges, HelixQA — MUST exercise the real, fully implemented HelixCode system against real infrastructure (real PostgreSQL, real Redis, real LLM endpoints, real containers, real captured devices). Production code (anything under `HelixCode/cmd/`, `HelixCode/applications/`, `HelixCode/internal/<pkg>/<file>.go` not ending `_test.go`) MUST NOT import from `HelixCode/internal/mocks/`.

**(B) 100% test-type coverage.** HelixCode's codebase MUST be covered by every supported test type the domain warrants:

- **Unit** — fast, isolated, mocks permitted per (A).

- **Integration** — multi-component, no mocks, real backing services.
- **End-to-end (E2E)** — full user-flow exercise on target topology.
- **Full automation** — orchestrated suites exercising every feature × platform combination (CONST-048 coverage ledger).
- **Security** — authn/authz boundaries, CONST-042 secret-leak scans, input-fuzzing, dependency-CVE scanning, threat-model verification.
- **DDoS** — request-flood resilience at advertised throughput tier.
- **Scaling** — horizontal + vertical scale behaviour under linear load growth.
- **Chaos** — controlled failure injection (network partition, process kill, disk full, clock skew).
- **Stress** — sustained load above advertised tier.
- **Performance** — latency / throughput / tail-latency invariants vs SLO baselines.
- **Benchmarking** — micro + macro suites with historical p95-drift detection.
- **UI** — visual-regression + DOM-state + interaction-flow coverage on every target platform's UI surface.
- **UX** — flow-correctness + accessibility + i18n + visual-cue ordering (§11.4.23 composition).
- **Challenges** — `vasic-digital/Challenges` submodule (at `./Challenges/`) fully incorporated; per-feature Challenge scripts with captured runtime evidence.
- **HelixQA** — `HelixDevelopment/HelixQA` submodule (at `./HelixQA/`) fully incorporated; ALL written test banks executed; full autonomous QA sessions run as part of release gates with captured wire evidence per check.

**Required dependency submodules** (recursive per CONST-047):

- Challenges — `git@github.com:vasic-digital/Challenges.git` — incorporated at `./Challenges/`.
- HelixQA — `git@github.com:HelixDevelopment/HelixQA.git` — incorporated at `./HelixQA/`.
- Any additional functionality submodules under `vasic-digital/*` / `HelixDevelopment/*` orgs that HelixCode depends on — incorporate rather than duplicate work the orgs already maintain.

Submodule pointers MUST be bumped to upstream HEAD in the SAME commit as any dependent cascade work (CONST-049 step 7). Pointer drift = CONST-050 violation.

**Cascade requirement:** This anchor (verbatim or by `CONST-050` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the release-gate layer. No escape hatch. See constitution submodule `Constitution.md` §11.4.27 for the full mandate.

# CONST-051: Submodules-As-Equal-Codebase + Decoupling + Dependency-Layout Mandate (cascaded from constitution submodule §11.4.28)

> Verbatim user mandate (2026-05-15): "*All existing Submodules in the project that we are controlling and belong to some our organizations (vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory - we can ALWAYS check dynamically using GitHub and GitLab CLIs) are equal parts of the project's codebase! We MUST work on that code as much as we do with main project's codebase! All on equal basis! Equally important! We MUST take it into the account, analyze it, extend it, create missing tests, do full testing of it, fill the gaps (if any), fix any issues that we discover or they pop-up, write and extend the documentation, user guides, manulas, diagrams, graphs, SQL definitions, Website(s) and all other relevant materials! We MUST NEVER modify Submodules to bring into them any project specific context since they all MUST BE ALWAYS fully decoupled, project not-aware, fully reusable and modular (by any other project(s)), completely testable! All Submodule dependencies that are used by Submodule MUST BE acessed from the root of the project! We MUST NOT have nested Submodule dependencies but accessing each from proper location from the root of the project - directly from project's root project_name/submodule_name or some more proper structure project_name/submodules/submodule_name!*"

Three cooperating invariants apply to every HelixCode-owned submodule (those whose upstream `origin` lives under `vasic-digital`, `HelixDevelopment`, `red-elf`, `ATMOSphere1234321`, `Bear-Suite`, `BoatOS123456`, `Helix-Flow`, `Helix-Track`, `Server-Factory`, or any subsequently authorised org):

**(A) Equal-codebase.** Every owned-by-us submodule is an **equal part** of HelixCode's codebase. The same engineering practice — analysis, extension, test creation, gap-filling, bug-fix, documentation (user manuals, guides, diagrams, graphs, SQL definitions, website pages, all materials) — applies to each owned submodule on equal basis. A round of work that improves only HelixCode's main while leaving an owned-submodule deficiency unaddressed is a CONST-051 violation, severity-equivalent to a §11.4 PASS-bluff at the project-scope layer. The §11.4.25 / CONST-048 coverage ledger MUST list every owned submodule as an in-scope target.

**(B) Decoupling / reusability.** Owned submodules MUST remain fully decoupled from HelixCode (and any other consuming project). No HelixCode-specific context, hardcoded paths, hostnames,

asset names, or runtime assumptions may be introduced into an owned submodule's source tree. When a submodule needs information from HelixCode, the honest path is configuration injection (env var, config file, constructor parameter) — never a hardcoded reach into the parent's tree. Every owned submodule MUST be project-not-aware, fully reusable, modular, and completely testable as a standalone repository.

**(C) Dependency-layout.** Every dependency that an owned submodule consumes MUST be accessible from HelixCode's root at one of two canonical paths:

- `<repo_root>/<submodule_name>/` (flat layout — current HelixCode layout for Challenges, HelixQA, Containers, Security, etc.)
- `<repo_root>/submodules/<submodule_name>/` (grouped layout — alternate)

**Nested own-org submodule chains are FORBIDDEN.** A submodule MUST NOT have its own `.gitmodules` entries pulling in further owned-by-us repos. Every dependency required by submodule X is added to HelixCode's root at the canonical path; X reaches it via documented import / SDK path / runtime resolver — never via its own nested submodule pointer. Third-party submodules (not under our orgs) are exempt — they MAY appear at any depth.

The owned-org list is dynamically discoverable at any time via `gh org list` / `glab` CLIs or the orgs' public APIs.

**Cascade requirement:** This anchor (verbatim or by `CONST-051` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the codebase-completeness layer. No escape hatch. See constitution submodule `Constitution.md` §11.4.28 for the full mandate (audit gates, mutation pairs, workflow integration).

| | | | | | | 4350384757760aabcf8d-f00be609fff98e9f1805 |
|---|---|---|---|---|---|---|

---

# Amendment Process

Constitution amendments require:

1. Written proposal with rationale
2. Challenge demonstrating the need
3. 72-hour review period
4. Approval by project architect
5. Update to all submodule governance files

---

*This Constitution is the supreme law of the HelixCode project. No code, test, or process may contradict it.*

<<<<<<< HEAD

# Article XI §11.9 — Anti-Bluff Forensic Anchor (Cascaded)

> Verbatim user mandate: "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"
>
> Operative rule: The bar for shipping is not "tests pass" but "users can use the feature." Every PASS in this codebase MUST carry positive runtime evidence captured during execution. No false-success results are tolerable.

## Bluff Taxonomy (cascaded from root CONSTITUTION.md)

- Wrapper bluff — assertions PASS but exit-code logic is buggy
- Contract bluff — advertises capability but rejects in dispatch
- Structural bluff — file exists but doesn't contain working code
- Comment bluff — comment promises behavior code doesn't have
- Skip bluff — t.Skip() without SKIP-OK: # marker

## Submodule Decoupling & Reusability — Mandatory

**Status:** Mandatory. Non-negotiable.

**Rule:** This repository is a **shared submodule** consumed by multiple independent consumer projects. Its value depends on staying **fully decoupled and reusable**. No change in this repository may introduce coupling that breaks its standalone reusability for any consumer.

**Prohibited inside this repository:**

1. Hardcoding any specific consumer project's name, paths, platform list, version strings, release-naming conventions, branding, or feature names.
2. `import` / dependency on any consumer-project namespace, package, or build coordinate.

3. Embedding consumer-project-specific governance, rule numbering, or release cadence into this repository's `CONSTITUTION.md` / `CLAUDE.md` / `AGENTS.md`.
4. Assuming this repository is consumed by a particular CLI, build system, language toolchain version, or target architecture beyond what its public interface documents.

**Required inside this repository:**

1. All public surfaces (APIs, CLIs, configuration files, environment variables, scripts) MUST be expressed in terms of THIS repository's own domain — not any consumer's.
2. Governance MUST describe responsibilities and contract from THIS repository's perspective. Consumer projects appear as illustrative examples at most, never as load-bearing requirements.
3. Cross-project rules adopted from a consumer (such as a cross-platform impact mandate) MUST be phrased generically — "every consuming project's full platform matrix" — and never hardcode any single consumer's matrix.

**Why:** Repositories like this one have shipped changes in the past where one consumer's platform list, feature names, or rule numbering leaked into shared-repo governance — and then collided at merge time with another consumer's parallel work, leaving the repository unmergeable until manual conflict resolution stripped the consumer-specific text back out. Decoupling is the only mechanism that preserves this repository's value as shared infrastructure.

**Recursive scope:** any submodule this repository consumes inherits the same decoupling+reusability rule. Third-party upstream submodules that this repository merely vendors (e.g. open-source tools under a `tools/opensource/` tree, if present) are explicitly out of scope — we are not their owners.

---

# CONST-047 — Recursive Submodule Application Mandate (cascaded from root CONSTITUTION.md)

> Verbatim user mandate (2026-05-14): "*Make sure all work we do is applied ALWAYS to all Submodules we control under our organizations (vasic-digital and HelixDevelopment) fully recursively everywhere with full bluff-proofing and comprehensive documentation, user manuals and guides and full tests and Challenges coverage!*"

Every engineering deliverable produced for the main project MUST be applied — fully and recursively — to every owned submodule under the `vasic-digital` and `HelixDevelopment` GitHub

organizations. Each owned submodule (including this one) MUST receive in lockstep: (1) anti-bluff posture (CONST-035 / Article XI §11.9), (2) comprehensive documentation matching actual capabilities, (3) full tests + Challenges coverage with captured runtime evidence, (4) recursive propagation through nested submodules under the same orgs, (5) synchronized commits when meta-repo state advances this surface.

## See the root `CONSTITUTION.md` §CONST-047 for the full mandate. This anchor MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md.

## CONST-048: Full-Automation-Coverage Mandate (cascaded from constitution submodule §11.4.25)

> Verbatim user mandate (2026-05-15): "*Make sure that every feature, every functionality, every flow, every use case, every edge case, every service or application, on every platform we support is covered with full automation tests which will confirm anti-bluff policy and provide the proof of fully working capabilities, working implementation as expected, no issues, no bugs, fully documented, tests covered! Nothing less than this does not give us a chance to deliver stable product! This is mandatory constraint which MUST BE respected without ignoring, skipping, slacking or forgetting it!*"

No feature / functionality / flow / use case / edge case / service / application on any supported platform of this submodule is deliverable until covered by automation tests proving six invariants: (1) anti-bluff posture with captured runtime evidence (CONST-035); (2) proof of working capability end-to-end on target topology; (3) implementation matching documented promise; (4) no open issues/bugs surfaced; (5) full documentation in sync; (6) four-layer test floor (pre-build + post-build + runtime + paired mutation).

**Cascade requirement:** This anchor (verbatim or by CONST-048 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-048 and constitution submodule `Constitution.md` §11.4.25 for the full mandate.

# CONST-049: Constitution-Submodule Update Workflow Mandate (cascaded from constitution submodule §11.4.26)

> Verbatim user mandate (2026-05-15): "*Every time we add something into our root (constitution Submodule) Constitution, CLAUDE.MD and AGENTS.MD we MUST FIRST fetch and pull all new changes / work from constitution Submodule first! All changes we apply MUST BE commited and pushed to all constitution Submodule upstreams! In case of conflict, IT MUST BE carefully resolved! Nothing can be broken, made faulty, corrupted or unusable! After merging full validation and verification MUST BE done!*"

Before ANY modification to `constitution/{Constitution,CLAUDE,AGENTS}.md` in the parent project, the agent or operator MUST execute the 7-step pipeline: (1) fetch + pull first inside the constitution submodule worktree; (2) apply the change with §11.4.17 classification + verbatim mandate quote; (3) validate (meta-test + no merge-conflict markers + cross-file consistency); (4) commit + push to EVERY configured upstream of the constitution submodule (governance files only — never `git add -A`); (5) careful conflict resolution preserving union of governance content (force-push forbidden per CONST-043 / §9.2); (6) post-merge `git submodule update --remote --init` + re-run cascade verifier (CONST-047); (7) bump consuming project's `.gitmodules` pointer to the new constitution HEAD in the SAME commit as cascade work.

**Cascade requirement:** This anchor (verbatim or by CONST-049 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-049 and constitution submodule `Constitution.md` §11.4.26 for the full mandate.

# CONST-050: No-Fakes-Beyond-Unit-Tests + 100%-Test-Type-Coverage Mandate (cascaded from constitution submodule §11.4.27)

> Verbatim user mandate (2026-05-15): "*Mocks, stubs, placeholders, TODOs or FIXMEs are allowed to exist ONLY in Unit tests! All other test types MUST interract with real fully implemented System! No fakes, empty implementations or bluffing is allowed of any kind! All codebase of the project MUST BE 100% covered with every supported test type: unit*

> *tests, integration tests, e2e tests, full automation tests, security tests, ddos tests, scaling tests, chaos tests, stress tests, performance tests, benchmarking tests, ui tests, ux tests, Challenges (fully incorporating our Challenges Submodule — https://github.com/vasic-digital/Challenges). EVERYTHING MUST BE tested using HelixQA (fully incorporating HelixQA Submodule — https://github.com/HelixDevelopment/HelixQA). HelixQA MUST BE used with all possible written tests suites (test banks) for every applications, service, platform, etc and execution of the full HelixQA QA autonomous sessions! All required dependency Submodules MUST BE added into the project as well (fully recursive!!!).*"

Two cooperating invariants:

**(A) No-fakes-beyond-unit-tests.** Mocks, stubs, fakes, placeholders, `TODO`, `FIXME`, "for now", "in production this would", or empty-implementation patterns are PERMITTED only in unit-test sources. Every other test type — integration, E2E, full automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges, HelixQA — MUST exercise this submodule's real, fully implemented system against real infrastructure. Production code MUST NOT import mock paths.

**(B) 100% test-type coverage.** Codebase MUST be covered by every supported test type the domain warrants: unit, integration, E2E, full-automation, security, DDoS, scaling, chaos, stress, performance, benchmarking, UI, UX, Challenges (vasic-digital/Challenges submodule fully incorporated), HelixQA (HelixDevelopment/HelixQA submodule fully incorporated, with full autonomous QA sessions executing every registered test bank with captured wire evidence).

**Required dependency submodules** (recursive per CONST-047): Challenges + HelixQA + any other functionality submodules under vasic-digital/HelixDevelopment orgs this submodule depends on.

**Cascade requirement:** This anchor (verbatim or by CONST-050 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-050 and constitution submodule `Constitution.md` §11.4.27 for the full mandate.

# CONST-051: Submodules-As-Equal-Codebase + Decoupling + Dependency-Layout Mandate (cascaded from constitution submodule §11.4.28)

> Verbatim user mandate (2026-05-15): "*All existing Submodules in the project that we are controlling and belong to some our organizations (vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory - we can ALWAYS check dynamically using GitHub and GitLab CLIs) are equal parts of the project's codebase! We MUST work on that code as much as we do with main project's codebase! All on equal basis! Equally important! ... We MUST NEVER modify Submodules to bring into them any project specific context since they all MUST BE ALWAYS fully decoupled, project not-aware, fully reusable and modular (by any other project(s)), completely testable! All Submodule dependencies that are used by Submodule MUST BE acessed from the root of the project! We MUST NOT have nested Submodule dependencies but accessing each from proper location from the root of the project - directly from project's root project_name/submodule_name or some more proper structure project_name/submodules/submodule_name!*"

Three cooperating invariants apply to every owned-by-us submodule (orgs: vasic-digital, HelixDevelopment, red-elf, ATMOSphere1234321, Bear-Suite, BoatOS123456, Helix-Flow, Helix-Track, Server-Factory, plus any subsequently authorised org — discoverable via `gh org list` / `glab`):

**(A) Equal-codebase.** This submodule is an EQUAL part of every consuming project's codebase. The consuming project's engineering practice — analysis, extension, test creation, gap-filling, bug-fix, documentation (user manuals, guides, diagrams, graphs, SQL definitions, website pages, all materials) — applies to this submodule on equal basis. Coverage ledgers (CONST-048) list this submodule as an in-scope target.

**(B) Decoupling / reusability.** This submodule MUST remain fully decoupled from any specific consuming project. NEVER inject project-specific context (hardcoded paths, hostnames, asset names, naming schemes). Stay project-not-aware, reusable, modular, completely testable as a standalone repository. When parent-project info is needed, use configuration injection (env var, config file, constructor parameter) — never a hardcoded reach.

**(C) Dependency-layout.** Any dependency this submodule consumes MUST be accessible from the consuming project's root at `<root>/<name>/` or `<root>/submodules/<name>/`. **Nested own-org sub-**

**module chains are FORBIDDEN** — this submodule MUST NOT have its own `.gitmodules` entries pulling in further owned-by-us repos. Third-party submodules are exempt.

# Cascade requirement: This anchor (verbatim or by CONST-051 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-051 and constitution submodule `Constitution.md` §11.4.28 for the full mandate.

4350384757760aabcf8d-f00be609fff98e9f1805

## CONST-052: Lowercase-Snake_Case-Naming Mandate (cascaded from constitution submodule §11.4.29)

> Verbatim user mandate (2026-05-15): "*naming convention for Submodules and directories (applied deep into hierarchy recursively) - all directories and Submodules MSUT HAVE lowercase names with space separator between the words of '_' character (snake-case)! All existing Submodules and directories which are not following this rule MUST BE renamed! However, since this will most likely break some of*

> *the functionalities renaming we do MUST BE applied to all references to particular Submodule or directory! ... There MUST BE reasonable exceptions for this rules - source code for programming languages or Submodules which apply different naming convention - Android, Java, Kotlin and others. ... Upstreams directory which all of our projects and Submodules have MUST BE renamed to the lowercase letters too, however root project containing the install_upstreams system command (it is exported in out paths in our .bashrc or .zshrc) MUST BE updated to fully work with both Upstreams and upstreams directory. ... NOTE: Rules lowercase / snake-case do apply to all project files as well and references to it and from them!"*

<<<<<<< HEAD Every directory, submodule, and file in this submodule MUST use lowercase snake_case names. Existing non-compliant names MUST be renamed atomically with updates to every reference (configs, docs, source-code imports, governance files). Reference drift after rename = CONST-052 violation of equal severity to the rename itself.

**Common-sense exceptions (technology-preserving):** language-mandated case for Java/Kotlin/Android/Apple/C#/Swift INSIDE language-roots; vendor/upstream third-party submodules keep upstream names; build artefacts (`node_modules`, `__pycache__`, `.git`, `target`, `build`, `bin`) keep tool-mandated names. The test "does renaming break the technology?" trumps the rule.

**`Upstreams/` → `upstreams/` transition:** the constitution submodule's `install_upstreams.sh` (exported via `.bashrc`/`.zshrc`) supports BOTH directory layouts; lowercase wins when both present.

**Test coverage of renames** (per CONST-050(B)): regression test for reference resolution + full test-type matrix run + anti-bluff wire-evidence captured.

# Cascade requirement: This anchor (verbatim or by CONST-052 ID reference) MUST remain in this submodule's CONSTITUTION.md, CLAUDE.md, and AGENTS.md, and propagate recursively to any nested owned-by-us submodule. See parent project's `CONSTITUTION.md` §CONST-052 and constitution submodule `Constitution.md` §11.4.29 for the full mandate.

Every directory, submodule, and file in HelixCode MUST use lowercase snake_case names. Existing non-compliant names (`HelixCode/`, `Challenges/`, `Containers/`, `HelixAgent/`, `HelixQA/`, `Security/`, `Github-Pages-Website/`, `Upstreams/`, `Dependencies/`, etc.) MUST be renamed as part of the phased migration opened by this clause. Every reference (configs, docs, links, source-code imports, governance files) MUST be updated atomically with the rename — reference drift after a rename is a CONST-052 violation of equal severity to the rename itself.

**Common-sense exceptions (technology-preserving):** language-mandated case for Java/Kotlin/Android/Apple/C#/Swift INSIDE the language root (submodule root follows our convention; subtree follows language convention); vendor/upstream third-party submodules keep upstream names; build artefacts (`node_modules`, `__pycache__`, `.git`, `target`, `build`, `bin`) keep tool-mandated names. The test "does renaming break the technology?" trumps the rule.

**`Upstreams/` → `upstreams/` transition:** the constitution submodule's `install_upstreams.sh` (exported via `.bashrc`/`.zshrc`) supports BOTH `Upstreams/` and `upstreams/` directory layouts (commit `45d3678` of the constitution submodule); lowercase wins when both present.

**Test coverage of renames** (per CONST-050(B)): every rename batch ships with (i) regression test verifying every reference now resolves, (ii) full test-type matrix run post-rename, (iii) anti-bluff wire-evidence captured.

**Phased execution** per the operator's explicit instruction: comprehensive brainstorming → phase-divided plan → fine-grained tasks/subtasks → every change covered by every applicable test type. §11.4.20 subagent delegation for cross-cutting rename sweeps.

**Cascade requirement:** This anchor (verbatim or by `CONST-052` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the reference-integrity layer. No escape hatch beyond the common-sense exceptions enumerated above. See constitution submodule `Constitution.md` §11.4.29 for the full mandate.

| | | | | | | 4350384757760aabcf8d-f00be609fff98e9f1805 |
|---|---|---|---|---|---|---|

# CONST-053: .gitignore + No-Versioned-Build-Artifacts Mandate (cascaded from constitution submodule §11.4.30)

> Verbatim user mandate (2026-05-15): "*every project module, every Submodule, every servcie and apolication MUST HAVE proper .gitignore file! We MUST NOT git version build artifacts, cache files, tmp files, main .env file(s) or any files containing sensitive data, API keys or token! Any build derivate which we can recreate by executing proper mechanism for generating MUST NOT be versioned! We MUST pay attention what is going to be commited every time we are preparing to execute commit! If any violetion is detected it MUST be fixed before commit is executed!*"

Every project module, owned-by-us submodule, service, and application MUST ship a proper `.gitignore`. Forbidden-from-version-control classes:

1. **Build artefacts**: `/bin/`, `/build/`, `/dist/`, `/out/`, `target/`, `*.exe`, `*.dll`, `*.so`, `*.dylib`, `*.a`, `*.o`, `*.class`, `*.pyc`, generator-produced files when the generator is committed.

2. **Cache files**: `__pycache__/`, `.pytest_cache/`, `.mypy_cache/`, `.ruff_cache/`, `node_modules/`, `.next/`, `.cache/`, `.gradle/`, `.terraform/`, language-server caches.
3. **Temp files**: `*.tmp`, `*.swp`, `*~`, `.DS_Store`, `Thumbs.db`, `*.orig`, `*.rej`.
4. **Sensitive-data files**: `.env`, `.env.*` (allow `.env.example` placeholder only — no real secrets even as examples), `*.pem`, `*.key`, `*.crt`, `id_rsa*`, `id_ed25519*`, `.netrc`, `secrets/`, `api_keys.sh`.
5. **Generated reports/logs**: `*.log`, `coverage.out`, `htmlcov/`, runtime captures unless reference assets.
6. **OS/IDE personal state**: `.idea/`, `.history/`, `.vscode/` (except shared settings).

**Anti-bluff invariant**: `.gitignore` line alone is not sufficient — no file matching the forbidden patterns may be CURRENTLY TRACKED. A tracked `*.log` despite the ignore-line is a violation of equal severity to no ignore-line at all.

**Pre-commit attention**: every commit author (human OR agent) MUST inspect `git diff --staged` + `git status` BEFORE executing the commit. Forbidden-class hits abort the commit until fixed (unstage, add to `.gitignore`, scrub if already-tracked). Gate `CM-GITIGNORE-PRECOMMIT-AUDIT` + paired mutation.

**Secret-leak intersection (CONST-042 / §11.4.10):** a `.env` leak is BOTH a CONST-053 and a CONST-042 violation; rotation + post-mortem required.

**Recreatable-content test**: if a documented mechanism regenerates the file from sources, it is a build derivative and MUST be ignored. The committed sources MUST include the generator.

**Cascade requirement:** This anchor (verbatim or by `CONST-053` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the repository-hygiene layer. See constitution submodule `Constitution.md` §11.4.30 for the full mandate.

# CONST-054: Submodule-Dependency-Manifest Mandate (cascaded from constitution submodule §11.4.31)

> Verbatim user mandate (2026-05-15): "*We MUST HAVE mechanism for each Submodule to determine / know what are its Submodule dependencies so new projects or palces we are incorporate them can add these Submodules to the project root and make them available! Suggested idea is configuration file with expected Submodules Git ssh urls per-*

> *haps? New project can read it, and recursively add each Submodule to the root of the project and install / expose it to veryone.*"

Every owned-by-us submodule MUST ship `helix-deps.yaml` at its root declaring its own-org dependencies. Schema: `schema_version`, `deps: [{name, ssh_url, ref, why, layout: flat|grouped}]`, `transitive_handling.{recursive,conflict_resolution}`, `language_specific_subtree`. Tooling: `incorporate-submodule <ssh-url>` adds the submodule at the parent project's canonical path (CONST-051(C)), reads `helix-deps.yaml`, recurses for each declared dep, aborts on conflicting refs, emits `<root>/.helix-manifest.yaml` audit record.

Anti-bluff guarantee: every manifest paired with a Challenge that bootstraps a throwaway consuming project, runs `incorporate-submodule`, asserts produced layout matches the manifest, runs the submodule's own tests against the bootstrapped layout, captures wire evidence per §11.4.2. A manifest without this proof is a CONST-054 violation.

§11.4.31 / CONST-054 is the **operational complement** of CONST-051(C): nested own-org submodule chains are FORBIDDEN — manifests are the bridge that lets consumers reconstruct the dependency graph at the parent root.

**Cascade requirement:** This anchor (verbatim or by `CONST-054` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to §11.4 PASS-bluff at the dependency-graph layer. See constitution submodule `Constitution.md` §11.4.31 for the full mandate.

# CONST-055: Post-Constitution-Pull Validation Mandate (cascaded from constitution submodule §11.4.32)

> Verbatim user mandate (2026-05-15): "*Every time we fetch and pull new changes on constitution Submodule we MUST process the whole project and all Submodule (deep recursively) for validation and verification taht every single rule or mandatory constraint is followed and respected! If it is not, IT MUST BE!*"

Whenever a project's constitution submodule is fetched + pulled with any content change, the project MUST run `scripts/verify-all-constitution-rules.sh` BEFORE the new constitution HEAD is treated as canonical for any other work. The sweep re-runs the governance-cascade verifier AND every implementable rule gate (CONST-053 `.gitignore` audit, CONST-051(C) nested-own-org-chain audit, CONST-052 case audit, CONST-050(A) mock-from-pro-

duction audit, CONST-035 anti-bluff smoke, etc.) against the post-pull tree. Failures populate the project's Issues tracker per §11.4.15 (Status: `Reopened`, Type: `Bug`); closure requires positive-evidence per §11.4.

Pull-time invocation: `git submodule update --remote constitution` triggers the sweep automatically (post-update hook OR commit-wrapper invocation). Operator-explicit manual invocation also available.

Anti-bluff: the sweep's own meta-test (paired mutation per §1.1) plants a known violation of each enforced gate and asserts the sweep reports FAIL for the planted gate. A sweep that exits PASS without running every implementable gate is a CONST-055 violation.

CONST-055 is the **enforcement engine** for every other §11.4.x and CONST-NNN rule — without it, new rules cascade as anchors but never get enforced.

**Cascade requirement:** This anchor (verbatim or by `CONST-055` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to §11.4 PASS-bluff at the constitutional-enforcement layer. See constitution submodule `Constitution.md` §11.4.32 for the full mandate.

# CONST-056: Mandatory install_upstreams on clone/add Mandate (cascaded from constitution submodule §11.4.36)

> Verbatim user mandate (2026-05-15): "*Every Submodule or Git repository we add or clone MUST BE upstreams installed using Upstreamable utility which MUST BE available through exported paths of the host system (in .bashrc or .zhrc) using install_upstreams command executed from the root of the cloned (added) repository - only if in it is Upstreams or upstreams directory present with bash script files (recipes) for all repository's upstreams!*"

Every clone / add of a Git repository under HelixCode MUST be followed by `install_upstreams` invocation from the repository's root IF its tree contains `upstreams/` (or legacy `Upstreams/` per CONST-052 transition) populated with `*.sh` recipe files. The utility (installed on operator's `PATH` via `.bashrc`/`.zshrc`; implementation in the constitution submodule's `install_upstreams.sh` — already sup-

ports BOTH directory names since constitution commit `45d3678`) reads the recipe files, configures every declared upstream as a named git remote, and fans out `origin` push URLs.

Skipping the invocation when `upstreams/` is present silently breaks §2.1 (multi-upstream push is the norm) — the next push lands on only one upstream. Gate `CM-INSTALL-UPSTREAMS-ON-CLONE` + paired mutation. Automation: the future `incorporate-submodule` per CONST-054 auto-invokes; manual invocation supported. Pre-commit check: `git remote -v | grep -c push` reports expected count.

**Cascade requirement:** This anchor (verbatim or by `CONST-056` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.36 for the full mandate.

# CONST-057: Type-aware Closure-Status Vocabulary (cascaded from constitution submodule §11.4.33)

Every project tracking work items by Type per §11.4.16 MUST close them with the Type-appropriate terminal `**Status:**` value, drawn from this 3-element closed map:

| Item `**Type:**` | Closure `**Status:**` value |
|---|---|
| `Bug` | `Fixed (→ Fixed.md)` |
| `Feature` | `Implemented (→ Fixed.md)` |
| `Task` | `Completed (→ Fixed.md)` |

The `(→ Fixed.md)` suffix is preserved across all three so the existing migration-discipline tooling (atomic Issues.md → Fixed.md move per §11.4.19) keeps working without per-Type branching. Generators (`generate_issues_summary.sh`, `generate_fixed_summary.sh`, the §11.4.23 colorizer) MUST treat the three terminal values as semantically equivalent (all "closed, positive evidence captured") while preserving the literal in the emitted document.

Closing a `Feature` with `Fixed (→ Fixed.md)` or a `Task` with `Implemented (→ Fixed.md)` is a CONST-057 violation. Gate `CM-CLOSURE-VOCAB-TYPE-AWARE` walks every Fixed.md heading + every Issues.md heading whose `**Status:**` is one of the three terminal values and asserts the Status-Type match. Composes with §11.4.15 / §11.4.16 / §11.4.19 / §11.4.23.

**Cascade requirement:** This anchor (verbatim or by `CONST-057` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.33 for the full mandate.

# CONST-058: Reopened-Source Attribution Mandate (cascaded from constitution submodule §11.4.34)

Every Issues.md (or equivalent project tracker) heading whose `**Status:**` is `Reopened` MUST carry, within 8 non-blank lines of the heading, a `**Reopened-Details:**` line capturing four sub-facts:

- **By:** `AI` or `User` (source-of-truth observer who flipped the status). `AI` covers in-loop reopens (test failure, gate regression, captured-evidence retrospect). `User` covers operator-side observations (manual testing, end-user report, design reconsideration).
- **On:** ISO date (`YYYY-MM-DD`).
- **Reason:** one-line cause classification — chosen from the closed vocabulary `{ test-failed | manual-testing-detected | captured-evidence-contradicts | end-user-report | cycle-rediscovered | design-reconsidered }`. Other values permitted with explicit `Reason: <free text>` annotation but the closed list MUST be tried first.
- **Evidence:** path to or short description of the captured artefact justifying the reopen — log file, recording, gate failure ID, operator quote, etc. Reopens without evidence are §11.4.6 / §11.4.7 violations (demotion from Fixed requires captured evidence under the conditions that re-exposed the defect).

The Issues_Summary.md Status column MUST distinguish the four `Reopened` sub-states by source so a sweep query for "reopens by AI in the last 30 days" is mechanically possible. Suggested column rendering: `Reopened (AI: test-failed)` vs `Reopened (User: manual-testing)`. Gate `CM-ITEM-REOPENED-DETAILS` mirrors `CM-ITEM-OPERATOR-BLOCKED-DETAILS` (§11.4.21 walk pattern). Composes with §11.4.6 / §11.4.7 / §11.4.15 / §11.4.21.

**Cascade requirement:** This anchor (verbatim or by `CONST-058` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.34 for the full mandate.

# CONST-059: Canonical-Root Inheritance Clarity (cascaded from constitution submodule §11.4.35)

The **constitution submodule's** three files (`constitution/Constitution.md`, `constitution/CLAUDE.md`, `constitution/AGENTS.md`) ARE the **canonical root** (also called the **parent** files). They contain only universal rules per §11.4.17.

The consuming project's **repository-root files** (`<project-root>/CLAUDE.md`, `<project-root>/AGENTS.md`, optionally `<project-root>/Constitution.md`) are **consumer extensions**. They MUST start with the inheritance pointer (either the Claude-Code native `@constitution/CLAUDE.md` import or the portable `## INHERITED FROM constitution/CLAUDE.md` heading). They contain only project-specific rules per §11.4.17.

**When in doubt about which file to edit:** universal rule → constitution submodule's file; project-specific rule → consumer's file. Default consumer-side when uncertain (§11.4.17 — narrower scope is cheap to widen).

**Terminology:** "the parent CLAUDE.md" / "the root Constitution" → constitution-submodule file at `constitution/<filename>`; "the project CLAUDE.md" / "this project's AGENTS.md" → consumer-side file at `<project-root>/<filename>`.

**No silent demotion or silent promotion.** Moving a rule between layers MUST be a visible commit — `git mv` of a section if it's a clean clone, or explicit `Lifted from <project> to constitution per §11.4.35` / `Demoted from constitution to <project> per §11.4.35` commit-message annotation.

Gate `CM-CANONICAL-ROOT-CLARITY` verifies (a) consumer's `CLAUDE.md` opens with the inheritance pointer, (b) constitution submodule's three files are present at the expected path, (c) no `## INHERITED FROM` block in the constitution submodule's own files (those ARE the source-of-truth, not consumers). Composes with §11.4.17.

**Cascade requirement:** This anchor (verbatim or by `CONST-059` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.35 for the full mandate.

# CONST-060: Fetch-before-edit Mandate (cascaded from constitution submodule §11.4.37)

> Verbatim user mandate (2026-05-15): "*Make sure that feedback_fetch_before_edit memory rule is part of our constitution Submodule - the root Consitution, AGENTS.MD and CLAUDE.MD. Validate and verify that Proejct-Toolkit and all Submodules do inherit all of them! Follow the constitution Submodule documentation for details.*"

The FIRST git-touching action of every session, on every consuming project (owned or third-party), MUST be:

```
git fetch --all --prune
git log --oneline HEAD..@{u}
git submodule foreach --recursive 'git fetch --all --prune --
        quiet'
```

If `HEAD..@{u}` is non-empty, integrate the upstream changes BEFORE any local edit. Acting on stale local state produces three failure modes documented in the originating §11.4.37 incident (multi-agent / parallel-session work): (1) **redundant work** — the agent re-does what a parallel session already finished, (2) **false confidence** — completion reports for already-done work, (3) **divergent history** — duplicate sibling commits that double the conflict surface on next push.

**Anti-bluff invariant**: the fetch+log check MUST produce captured evidence — the actual `HEAD..@{u}` output, even if empty. Skipping the check on the basis of "I just fetched" or "nothing could have changed in the last N minutes" is a §11.4.6 (no-guessing) violation: the remote state is not knowable without a fetch.

**Cascade requirement**: This anchor (verbatim or by `CONST-060` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to §11.4 PASS-bluff at the parallel-session-coordination layer. See constitution submodule `Constitution.md` §11.4.37 for the full mandate. <<<<<<< HEAD

# Anti-Bluff End-User Quality Guarantee (Escalated via HelixConstitution)

**Canonical authority:** `HelixConstitution/Constitution.md` §7.1 and §11.4. This section is this submodule's binding pointer to those universal clauses.

**Forensic anchor — verbatim operator mandate (2026-04-28):**

> "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product! This MUST BE part of Constitution of our project, its CLAUDE.MD and AGENTS.MD if it is not there already, and to be applied to all Submodules's Constitution, CLAUDE.MD and AGENTS.MD as well (if not there already)!"

**Operative rule:** the bar for shipping is **not** "tests pass" but **"users of any consuming project can use the feature."** Every PASS MUST carry positive runtime evidence captured during execution that the feature actually works end-to-end. Metadata-only PASS, configuration-only PASS, "absence-of-error" PASS, and source-grep-only PASS without runtime evidence are critical defects.

**Required minimum evidence per test category:**

| Category | Minimum evidence |
|---|---|
| Unit tests | Real inputs + mutation-verified assertions (stub-swap must cause FAIL) |
| Integration tests | Real subsystems only; mocks only at honest external boundaries |
| E2E / Challenge tests | Per-test PASS/FAIL lines + log-file artefact path; RUNTIME layer mandatory |

**Decoupling constraint:** this rule applies to every consuming project's full platform matrix — specific platform names are not hardcoded here.

=======

> 4350384757760aabcf8d-f00be609fff98e9f1805

# CONST-061: Pre-Force-Push Merge-First Mandate (cascaded from constitution submodule §11.4.41)

> Verbatim user mandate (2026-05-17): "*make sure we bring everything from branches to our side before forc push is done! Afer everything is safely and fully merged and all potential conflicts (if any) resolved, then do force push! make*

> *sure nothing isnlost, broken or corrupted on bith sides! add these rules in our root Constitution, CLAUDE.MD, AGENTS.MD (constitution Submodule) if itnis not added already! Extremely important rules and mandatory constraints we MUST HAVE and fully respect!"*

Any force-push (`--force`, `--force-with-lease`, `+<ref>`, equivalent history-rewrite) authorised under CONST-043 MUST be preceded by a mechanical 4-step merge-first pipeline:

1. **Fetch every remote** — `git fetch --all --prune --tags` against origin + every upstream; capture output.
2. **Integrate every divergent commit locally** — rebase / merge / operator-confirmed cherry-pick per appropriate strategy for every non-empty `HEAD..<remote>/<branch>` range.
3. **Audit the integrated tree** — no conflict markers anywhere (`grep -rn '^<<<<<<< \|^=======$\|^>>>>>>> '` returns empty in governance + source + test files); no file silently dropped; previously-passing tests still pass; captured-evidence artefacts still validate.
4. **Force-push** — only after steps 1-3 produce clean integration evidence: `git push --force-with-lease` (NEVER `--force` alone unless authorised per §9.2 sub-clause 6).

**Two-gate composition with CONST-043.** §11.4.41 does NOT relax CONST-043's operator-approval requirement — it adds a SECOND mechanical gate. CONST-043 alone authorises a push that loses remote work; §11.4.41 alone risks pushing without operator awareness. Both required.

**Three failure modes prevented:** (a) remote-side content loss when parallel sessions land work between fetches; (b) stale-state acts when `--force-with-lease` reads stale local refs without prior fetch; (c) conflict-driven corruption when markers get committed verbatim (observed 2026-05-17 in helix_qa + containers governance files).

**Verification artefact**: every governed force-push emits a `docs/changelogs/<tag>.md` "Force-push merge-first audit" section capturing fetch output, per-remote divergence log, integration strategy, conflict-marker scan, test delta, push output with lease SHA, + CONST-043 authorisation quote. Gate `CM-FORCE-PUSH-MERGE-FIRST` + paired mutation.

**Cascade requirement:** This anchor (verbatim or by `CONST-061` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the remote-data-integrity layer. See constitution submodule `Constitution.md` §11.4.41 for the full mandate.

# CONST-068: Shell-script target-shell-parseability mandate (cascaded from constitution submodule §11.4.67)

> Verbatim user mandate (2026-05-19): "*any issue we spot must be fixed, bash scripts as well if they are broken!*" + "*Make sure that this is mandatory rule!*"

> Verbatim 2026-05-19 operator mandate: "*all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product!*"

Every committed shell script MUST be parseable by its target interpreter (`sh -n` for `/bin/sh`, `bash -n` for `/bin/bash`, etc.) AND MUST declare a shebang matching its actual syntax usage. Bash-only constructs (`>(...)`, `<(...)`, `[[ ]]`, `<<<`, arrays, `${var^^}`, etc.) used in scripts that may be invoked via `sh script.sh` MUST be wrapped in `eval` so the parser sees only a string (target shells like mksh parse the entire script before executing — runtime guards cannot save a parse-time rejection). Honest shebangs only: `#!/bin/bash` only if bash actually expected; `#!/bin/sh` requires POSIX-clean body. Fix at source per §11.4.1, never at callsites. Composes with §11.4.1 / §11.4.4 / §11.4.6 / §11.4.50 / §11.4.51. Pre-build gate `CM-SCRIPT-TARGET-SHELL-PARSEABLE` runs `sh -n` on every in-scope script. No escape hatch — no `--skip-parseability-check`, `--bash-only-script`, `--runtime-guard-suffices` flag.

**Cascade requirement:** This anchor (verbatim or by `CONST-068` ID reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. See constitution submodule `Constitution.md` §11.4.67 for the full mandate.

# §11.4.68 — Positive Sink-Side / Downstream Evidence Mandate (cascaded from constitution submodule §11.4.68)

> Verbatim user mandate (2026-05-20): "*We still do not hear any audio played from D3 device! Arvus Web Dashboard when we play music from D3 shows nothing for Codec In Use! This MUST BE investigated and fixed! How come we passed the tests with Arvus validation? What were values for*

> *the Codec In Use field? Empty means nothing! This is not working! It MUST BE FIXED, TESTED AND VERIFIED WITH FULL AUTOMATION TESTING ASAP!!!*"

A test that asserts audio or video routing PASS MUST capture and verify **positive sink-side or downstream evidence** — never config-only, never metadata-only, never PCM-open-state-only. At least one of the closed enumeration MUST be captured for every audio/video routing PASS: (1) sink-side codec-state with non-empty Codec-In-Use matching the expected codec regex; (2) strictly-positive PCM frames-written delta from `/proc/asound/.../status hw_ptr`; (3) ALSA ELD/EDID-Like-Data showing negotiated channel count + format; (4) ffprobe-on-captured-mp4 with non-zero frame count + expected codec/resolution/fps; (5) recording-analyzer event match per §11.4.2/§11.4.5; (6) tinycap RMS amplitude above the line-level floor. Empty / `<unreachable>` / `<N.E.>` / `<None>` placeholders are NOT positive evidence; a missing-but-required sink is `OPERATOR-BLOCKED` (release-blocker), never SKIP, never PASS. No escape hatch — no `--skip-sink-evidence`, `--allow-empty-codec`, `--sink-unreachable-is-pass`, `--metadata-only-suffices` flag exists.

**Cascade requirement:** This anchor (verbatim or by `§11.4.68` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the sink-side-evidence layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.68 for the full mandate.

# §11.4.70 — Subagent-Driven Execution Is The Default (cascaded from constitution submodule §11.4.70)

> Verbatim user mandate (2026-05-20): "*Always do if possible Subagent-driven! Add this into our root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md. This should be the default choice ALWAYS!*"

When executing implementation plans (or any task-decomposed execution flow), the **default execution model is subagent-driven** per `superpowers:subagent-driven-development`. Inline execution is permitted ONLY when (a) the task is trivial AND fits a single sub-300-line edit, OR (b) the operator explicitly requests inline at brainstorm-handoff time. Subagent-driven is the default because it gives isolated context per task, naturally enforces two-stage review, is parallel-PWU compatible (§11.4.58), creates an anti-bluff seam (§11.4), and survives operator absence. No escape hatch — `--inline-execution-required`, `--no-subagents`, `--monolithic-execution`

are NOT permitted flags. Skipping subagent-driven for non-trivial work without recorded operator authorisation is itself a §11.4 PASS-bluff.

**Cascade requirement:** This anchor (verbatim or by `§11.4.70` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a §11.4 PASS-bluff at the execution-model layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.70 for the full mandate.

# §11.4.71 — Pre-Push Fetch + Investigate + Integrate Mandate (cascaded from constitution submodule §11.4.71)

> Verbatim user mandate (2026-05-20): "*before pushing changes to any upstream for any repository - main repo or Submodule, we MUST fetch and pull all changes. Once these are obtained WE MUST investigate what is different compared to head position we were on last time before fetching and pulling new changes! We MUST understand what is done and for what purpose, easpecially how that does affect our project and our System in general! Any mandatory changes or improvements required by fresh changes we just have brough in MUST BE incorporated, covered with all supported types of the tests which will produce as a result of its success execution REAL PROOFS of working for all componetns and functionalities covered and work fully in anti-bluff manner!*"

The everyday-push variant of §11.4.41. EVERY push (every repository — main + every submodule) MUST follow the 5-step cycle: (1) fetch all remotes (`git fetch --all --prune --tags`, capture stdout); (2) pull all upstream branches whose tip differs, resolving conflicts per consumer judgment (never auto-`--ours`/`--theirs`); (3) investigate the diff vs OUR previous HEAD — read EVERY foreign commit's body, understand what/why/how-it-affects-our-system; (4) integrate mandatory changes with §11.4.4(b) four-layer coverage + §11.4.43 TDD-fix discipline, every PASS carrying §11.4.5 captured-evidence (REAL PROOFS, not metadata-only); (5) only then push, verifying with `git ls-remote` post-push. No escape hatch — no `--skip-fetch`, `--no-investigate`, `--fast-push`, `--trust-upstream` flag.

**Cascade requirement:** This anchor (verbatim or by `§11.4.71` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a

§11.4 PASS-bluff at the push-discipline layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.71 for the full mandate.

## §11.4.72 — Audio Top-Priority Mandate (cascaded from constitution submodule §11.4.72)

> Verbatim user mandate (2026-05-20): "*Make sure all fixes for audio are always top priority in main working stream!*"

The conductor (main working stream — Claude Code session, AI agent, or human operator) MUST treat audio fixes as the highest-priority class on the serial dispatch queue. Any time the conductor faces a choice between dispatching an audio task vs a non-audio task on the SAME serial resource, the audio task wins. Parallel BACKGROUND subagents (research, refactors, infrastructure documentation) MAY run concurrently with audio work but do NOT preempt audio on the main-stream serial dispatch queue. No escape hatch — there is no "but this non-audio task is faster" or "but this research is more interesting" override; audio-stack regressions are user-perceptible and high-impact while research and refactors can wait.

**Cascade requirement:** This anchor (verbatim or by `§11.4.72` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a process violation at the dispatch-priority layer. **Canonical authority:** constitution submodule `Constitution.md` §11.4.72 for the full mandate.

## §11.4.73 — Main-Specification Document Versioning + Revision Discipline (cascaded from constitution submodule §11.4.73)

> Verbatim user mandate (2026-05-20): "*Make sure everything we add now in previous and upcoming requests IS ALWAYS applied to the main specification — if we have one. Since all these are not major changes we could increase Specification version per change for secondary version instead of the primary. Primary version MUST BE increased for much bigger levels of changes! Add this into root (constitution Submodule) Constitution.md, CLAUDE.md and AGENTS.md as mandatory rule / constraint applicable ONLY IF we have something like the main specification document or we do recognize something like the main specification document.*

> *Document MUST BE updated ALWAYS to follow the versioning rules we are appling here + revision and other properties we have!*"

Applies **only when a project recognises a main specification document**. When it does: (1) every additive operator requirement, refinement, or accepted recommendation MUST be applied to the spec before or as part of the implementing work; (2) spec versioning has two axes — *primary* (V1/V2/V3, bumped for major rewrites by explicit operator decision, old versions archived) and *secondary* (the §11.4.61 metadata-table `Revision` integer, bumped for every other change); (3) the metadata table MUST stay current (`Revision`, `Last modified`, `Status summary`, `Fixed`); (4) propagated copies of the rule MUST reference the active `specification.V<primary>.md`, not a stale archive; (5) on primary bump the old file moves to `<spec-dir>/archive/` with `Status: superseded`. Classification: universal, applicable conditionally per the scope condition.

**Cascade requirement:** This anchor (verbatim or by `§11.4.73` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a release blocker when a project has a main spec and lets it drift. **Canonical authority:** constitution submodule `Constitution.md` §11.4.73 for the full mandate.

# §11.4.74 — Submodule-Catalogue-First Discovery + Extend-Don't-Reimplement (cascaded from constitution submodule §11.4.74)

> Verbatim user mandate (2026-05-20): "*We MUST ALWAYS check which already developed features / functionalities do exist as a part of our comprehensive Submodules catalogue located in vasic-digital and HelixDevelopment organizations on GitHub and GitLab both! Project MUST BE aware of all its existence so we do not implement same things multiple times if they are already done as some of existing universal, reusable general development purpose Submodules! For any missing features that some Submodules we incorporate may be missing we MUST IMPLEMENT the properly and extend those Submodules furter! We do control all of the and we CAN and MUST maintain and extend the regularly! All development cycle rules we have MUST BE applied to them and fully respected!*"

Before scaffolding ANY new module, package, helper, or utility, the contributor (human or AI agent) MUST: (1) survey the canonical Submodule catalogue — `vasic-digital` and

`HelixDevelopment` on both GitHub AND GitLab; (2) inventory existing Submodules; (3) reuse before reimplement — if a Submodule provides the functionality (or 80%+ of it), add it as a Git submodule rather than write fresh; (4) extend in-place when 80%+ matches but features are missing — add the missing features TO THAT SUBMODULE (PR upstream + bump pointer), never as a duplicating consuming-project helper; (5) apply all development-cycle rules to those Submodules; (6) document the survey result in the feature's tracker entry with a `Catalogue-Check:` field (`reuse <org/repo>@<sha>` / `extend <org/repo>@<sha>` / `no-match <date>`). Classification: universal.

**Cascade requirement:** This anchor (verbatim or by `§11.4.74` reference) MUST appear in every owned submodule's `CONSTITUTION.md`, `CLAUDE.md`, and `AGENTS.md`. Severity-equivalent to a process violation; duplicate implementations landed without catalogue check are release blockers. **Canonical authority:** constitution submodule `Constitution.md` §11.4.74 for the full mandate.